Temco Platform

temco-editor · built Nov 10, 2026

temco-editor

The editor for a Markdown page: one module, one editor, three layers. It owns the surface and nothing about what a page *is* — that stays in `temco-docs`.

Identifiers

Extension id	`temcocontrols.temco-editor`
View type	`temcoEditor.markdown` — **Temco Editor**, `priority: "default"` for `*.md`
Commands	`temcoEditor.open` · `temcoEditor.openSource` · `temcoEditor.preview` · `temcoEditor.diff` · `temcoEditor.pageProperties`
Namespace	`temcoEditor`
Storage	none
Depends on	`temcocontrols.temco-docs` (its model, through `src/docs.ts`)

Why it exists

Writing a page in a text editor means thinking about pipes and asterisks; a WYSIWYG surface means writing the page. The editor used to live inside `temco-docs` as `temcoDocs.page`, next to the registry, the tree, the builds and the release report — 494 KB of documentation domain beside ~250 KB of editor. This module is that 250 KB, given its own front door.

It also replaced its own prototype: for a while there were two editors to compare. The comparison is over, the winning affordances are here — a **selection menu** for the marks, a **drag handle** beside a block, a **placeholder** on an empty page — and the prototype is deleted.

The three layers

```text
the surface      main.tsx · ribbon.tsx · reading.tsx      asks for named operations, rea
   ↕ contract    webview/markdown/contract.ts             PageState · PageActions · Mark
the engine       webview/markdown/engine/tiptap/          the only place a Tiptap packag
```

The rule is enforced by reading, not by trust: `grep -r "@tiptap" webview/markdown/*.tsx webview/shared/` returns nothing, `scripts/check-shortcuts.mts` reads the shortcuts out of `engine/tiptap/document.tsx` rather than assuming them, and replacing Tiptap means deleting one folder.

The host is the same shape from the other side: `src/` knows a document, a panel, an edit and a line ending, and no `Editor`, `Node`, `Mark`, `PluginKey` or `Transaction`. What a page *is* — its media folder, the version it describes, the pages a link may reach — it asks `temco-docs` for, through `src/docs.ts`.

What it owns

- the view type, the provider, and the commands that reach it;
- the bridge to the file: one `WorkspaceEdit` per save, the file's own line endings, and the ordered-write guard that keeps a caret still while two writes land out of order;
- the Markdown contract (`src/markdown.ts`) — the rules that make an unedited page open and save to the same bytes;
- the surface: the ribbon, the selection menu, the block handle, find, the outline rail, read mode, zoom, capture;
- its bundle and the one bundler step that produces it.

What it must not own

- **the registry, the pages model, the builds, the release report** — `temco-docs`;
- **where an image lands.** `media/`, the file name and the attachment record are the model's; this module supplies the bytes and the words and asks (`landImage`);
- **a second Markdown dialect.** The round trip is one implementation. If Tiptap's own serialiser would be better, that is a change to the contract, proved by `check-page` first.

What was learned the hard way

- **A task list was destroyed twice** — on open, because `marked` writes `<ul><li><input type="checkbox">` and Tiptap wants `ul[data-type="taskList"]`; on save, because `turndown-plugin-gfm` wants the `<input>` as the item's first child and Tiptap renders it inside a `<label>`. Both shapes are now accepted on the way in (`engine/tiptap/taskLists.mts`) and the checkbox is read wherever it sits on the way out (`src/markdown.ts`).

- **Tiptap merges attribute parsers over a rule's `getAttrs`**, so `checked` had to be parsed in `addAttributes`, not in the parse rule.
- **A missing bundle is a blank tab, not an error**: `bridge.ts` asks for `markdown.js` and `markdown.css` by name, a 404 on a script fires no `onerror`, and `compile-extension:<name>` cleans `out/` before `tsc` — which deletes the bundle. `check-surface.mts` loads the built file, so a stale or missing bundle fails the gate instead of the tab.

Gates

`npm run vscode:prepublish`, in order: `check-commands` · `check-fluent` · `check-types` (host and webview, separately) · `compile` · `check-page` (13 round-trip cases) · `check-task-lists` (a real editor, both shapes) · `check-outline` (29 cases, no browser) · `check-shortcuts` (every key the bar teaches is bound) · `build-webview` · `check-surface` (the built bundle in jsdom: it starts, draws the page, writes one edit back, keeps its checkboxes, and survives a round trip through Read mode).

From the repository root, `npm run compile-temco` builds the extensions **and** both webview bundles — the only correct way to rebuild after a `tsc` run, which cleans `out/` first.